Guidelines for Round 2 and Submission

- Select your problem statement by filling out the given form.
- If you are working on a self-defined problem statement, inform us in advance.
- You need to submit a PPT on the problem statement by the given deadline. (15th August)
- The PPT should strictly consist of 6 slides.
- The PPT should cover:
  1. Problem Statement
  2. Your Solution
  3. Innovation
  4. Tech Stack
  5. Implementation Plan
  6. Impact & Future Scope
- Add any visuals/ video demonstrations/wireframes if needed.

All the best!

-Team ECE Society

TECH-A-THON 4.0

TECH-A-BIT

PROBLEM STATEMENTS

AGRO-TECH AND RURAL DEVELOPMENT

TECH - PROBLEM STATEMENTS

PROBLEM STATEMENT 1 - [Type: Deep Learning + Web App] Build a crop disease detection web app for farmers.

Description: Farmers often notice a diseased leaf too late, after the infection has already spread. Build a simple web app where a user uploads a photo of a leaf, and a trained model predicts which disease it likely has, along with a basic remedy tip.

The solution could focus on one or more of the following:

- **Image Classification:** Train a small CNN, or fine-tune a ready-made model (MobileNet/ ResNet), on a public leaf-disease dataset (PlantVillage) to classify 3-5 common diseases.
- **Web Upload Page:** A basic web page/form where a user uploads an image and the backend returns the predicted disease with a confidence score.
- **Remedy Lookup:** Store a simple text tip for each disease class (e.g., in a small database or even a JSON file) and display it alongside the prediction.
- **History Page:** Show a simple list of a user's past uploads and predictions.

OPTIONAL Add-on Features:

- Show top-3 predictions instead of just one, in case the model is unsure.
- Multilingual remedy text.

PROBLEM STATEMENT 2 - [Type: Basic ML + Web App] Design a farm expense tracker with simple spend prediction for small farmers.

Description: Small farmers often track input costs (seeds, fertilizer, labor, equipment) only informally, and realize they've overspent only after the season is over. Build a web app to log farm expenses and income by category, and add a basic ML model that predicts next season's likely spend based on past seasons.

The solution could focus on one or more of the following:

- **CRUD Web App:** A web app (any stack) where a farmer can add/edit/delete transactions with a category (seeds/fertilizer/labor/equipment) and amount, with simple login.
- **Charts:** Show category-wise and season-wise totals using basic bar/pie charts.
- **Prediction:** Use simple linear regression on past season totals (per category) to estimate next season's spend. No need for time-series libraries --- plain regression on season number vs. amount is enough.
- **Budget Flag:** Let the farmer set a season budget per category, and show a red/green flag if the predicted spend crosses it.

OPTIONAL Add-on Features:

- Manually mark a transaction as "unusual" and just count/highlight repeated unusual entries (no need for an anomaly-detection model).
- Simple expense splitting between 2-3 co-farmers sharing a field.

PROBLEM STATEMENT 3 - [Type: Basic ML + Web App] Build a farm-scheme and advisory misinformation checker.

Description: Farmers receive a steady stream of WhatsApp forwards about government schemes, subsidies, and farming tips, and many of these are outdated or misleading. Build a web tool where a user pastes a message or headline, and a basic ML model predicts whether it looks likely to be misleading or genuine.

The solution could focus on one or more of the following:

- **Text Classifier:** Train a Logistic Regression or Naive Bayes model using TF-IDF features on a public fake-news dataset (adapted or relabeled with a small custom set of real vs. misleading agri-scheme messages).
- **Web Form:** A simple page where users paste text and get back a prediction ("likely genuine" / "likely misleading") with a confidence score.

- **Keyword Highlight:** Show the top few words that pushed the prediction one way or the other (TF-IDF weight is enough, no need for LIME/SHAP).
- **Simple History Log:** Store past checks in a table so users can revisit them.

OPTIONAL Add-on Features:

- Let users upvote/downvote if they think the prediction was right, and just log this feedback (no need to retrain live).
- Basic domain-name check against a small hardcoded list of verified government scheme portals.

PROBLEM STATEMENT 4 - [Type: Basic ML + Web App] Create a village farm-equipment and labor sharing board with simple matching.

Description: Small and marginal farmers often can't afford their own tractor, tiller, or sprayer, and rely on word-of-mouth or messy WhatsApp groups to find equipment or extra labor nearby. Build a web app where people post equipment or labor "available" or "needed" listings, and the system suggests possible matches based on shared details.

The solution could focus on one or more of the following:

- **Listings Web App:** Users can post an "available" or "needed" listing for equipment or labor with a title, description, category, date, and village/location.
- **Simple Matching Logic:** Compare posts using basic text similarity (e.g., shared keywords, or TF-IDF + cosine similarity between description text) to suggest likely matches --- no image matching needed.
- **Match Suggestions:** When a new post is added, show the top 3 possible matching posts from the opposite list.
- **Admin View:** A simple page listing all open and resolved requests.

OPTIONAL Add-on Features:

- Let users optionally attach a photo (just for display, not for ML matching).
- Simple category/date/village filters on the listings page.

PROBLEM STATEMENT 5 - [Type: Basic ML + Web App] Build a farmer-to-government-scheme eligibility matching tool.

Description: There are dozens of agricultural subsidy and welfare schemes, and small farmers rarely know which ones they actually qualify for. Build a web tool where a farmer fills in a simple

profile (land size, crop, state, category) and the tool ranks available schemes by how relevant they are to that profile.

The solution could focus on one or more of the following:

- **Profile & Scheme Text Input:** Let farmers fill a short profile form, and let an admin paste/ upload scheme description text (plain text is fine, no need for structured parsing).
- **Relevance Scoring:** Use TF-IDF + cosine similarity between profile keywords and scheme description text to produce a relevance score and rank schemes --- no embeddings/ transformers needed.
- **Farmer Dashboard:** A page where the farmer enters their profile and sees a ranked list of matching schemes.
- **Scheme Status Page:** A simple page listing scheme details and application links.

OPTIONAL Add-on Features:

- Highlight which eligibility keywords from the scheme were found/missing in the farmer's profile.
- Simple manual bookmark/save button for farmers.

PROBLEM STATEMENT 6 - [Type: Basic ML + Web App] Design a mandi (market) feedback sentiment dashboard for local farmers.

Description: Farmers selling produce at local mandis get scattered feedback from buyers and traders with no easy way to see overall sentiment. Build a dashboard that reads in buyer/trader comments and classifies them as positive or negative.

The solution could focus on one or more of the following:

- **Sentiment Model:** Use a ready-made simple sentiment tool (like TextBlob or VADER, both plug-and-play, no training needed) or train a basic Naive Bayes/Logistic Regression classifier on a small labeled review dataset.
- **Review Input:** A form or CSV upload where a farmer or cooperative adds buyer/trader comments.
- **Dashboard:** Show a simple chart of positive vs negative feedback over time, and a list of most common words in negative feedback.
- **Alert:** Flag when negative feedback crosses a set percentage in a week.

OPTIONAL Add-on Features:

- Basic keyword tagging (e.g., manually tag comments as "about price" / "about quality" and show sentiment split by tag).

- Simple reply-template suggestions for negative feedback (just static templates, no generation needed).

PROBLEM STATEMENT 7 - [Type: Basic ML + Web App] Build a village water-pump and irrigation slot availability predictor.

Description: In many villages, a shared water pump or irrigation channel gets crowded at certain hours, and farmers waste time walking over just to check if it's free. Build a simple system that predicts busy times using past usage patterns.

The solution could focus on one or more of the following:

- **Data Logging:** Manually log or simulate pump/channel usage (number of users) at a few time slots over a week or two.
- **Prediction Model:** Use basic linear regression or a decision tree to predict expected usage for a given hour and day of week (no need for real time-series models).
- **Dashboard:** Show each pump/channel's predicted usage as a simple green/yellow/red indicator with a basic chart by hour.
- **Manual Check-in:** Let farmers manually report current usage, adding to the dataset over time.

OPTIONAL Add-on Features:

- Simple notification (even just an on-page banner) when a chosen pump/channel is predicted to be free soon.
- A basic table view comparing predicted vs. actual usage accuracy.

PROBLEM STATEMENT 8 - [Type: Basic ML + Web App] Create a daily farm-activity log with crop health trend tracking.

Description: Farmers often notice a problem with their crop (wilting, pest signs, discoloration) only after it has already spread across the field. Build a simple, private journal app where a farmer writes a short daily note and picks a crop-condition tag for a given field, and the app tracks the crop's condition trend over time (never diagnosing anything --- just showing a trend and a gentle nudge to inspect the field if things decline for a while).

The solution could focus on one or more of the following:

- **Journal Web App:** A private page (with login) where a farmer writes a short entry and selects a condition tag (healthy/mild concern/pest seen/wilting, etc.) for a field or crop.

- **Sentiment/Tag Scoring:** Use a ready-made tool (TextBlob/VADER) to score the entry text --- no training required --- or simply use the farmer's selected condition tag directly.
- **Trend Dashboard:** A simple line chart of crop condition over the past days/weeks, viewable only by that farmer, per field.
- **Gentle Nudge:** If condition stays poor for several days in a row, show a supportive message suggesting a field inspection or a call to the local agriculture extension helpline --- nothing diagnostic.

OPTIONAL Add-on Features:

- A simple word-cloud of frequently used words in entries, for the farmer's own reflection.
- Weekly summary notification of a field's own trend.

PROBLEM STATEMENT 9 - [Type: Deep Learning + Web App] Design a face-recognition based attendance system for rural workfare labor sites with a supervisor dashboard.

Description: Manual roll-call at rural work sites (e.g., MGNREGA-style daily wage work) wastes time and can be gamed with proxy attendance, which affects fair wage payouts. Build a system where a camera captures worker faces at the site, a face-recognition model marks who's present, and the supervisor can view/manage the record on a simple dashboard.

The solution could focus on one or more of the following:

- **Face Recognition:** Use a ready-made face recognition library (e.g., Python's face_recognition package, built on a pretrained deep model) to enroll a few worker photos and match faces from a captured image --- no need to train a model from scratch.
- **Attendance Logging:** When a face is matched above a confidence threshold, log the worker and timestamp automatically.
- **Supervisor Dashboard:** A simple web page for the supervisor to start a session, view/edit the attendance list, and export it as CSV for wage records.
- **Worker View:** A basic page where a worker can log in and see their own attendance/wage-days record.

OPTIONAL Add-on Features:

- A manual-review list for low-confidence matches instead of silently getting it wrong.
- Low-attendance alert for workers below a cutoff, flagged to the supervisor.

PROBLEM STATEMENT 10 - [Type: Basic ML + Web App] Build a personalized crop and agri-input recommendation web app.

Description: Deciding what to grow next or which local agri-input shop to buy seeds/fertilizer from is mostly word-of-mouth with no real personalization. Build a simple web app where crops or agri-input shops are listed with basic tags (soil type, season, budget, crop type), farmers rate ones they've tried, and the app recommends new options based on shared tags with ones they rated highly.

The solution could focus on one or more of the following:

- **Listings & Ratings App:** A web app to list crops or agri-input shops with basic attributes and let farmers rate ones they've tried (1-5 stars).
- **Simple Recommendation Logic:** For each farmer, find their highest-rated crop/shop's tags, and recommend other options sharing the most tags (basic tag-overlap scoring --- no matrix factorization needed).
- **Filter & Search Page:** Let farmers filter listings by soil type, season, and budget.
- **Basic Map/List View:** Show recommended options in a simple list (map view optional, and can just use static coordinates, no live GPS needed).

OPTIONAL Add-on Features:

- "Because you liked X" label next to each recommendation.
- Simple photo upload for listings (display only).

CORE - PROBLEM STATEMENTS

PROBLEM STATEMENT 1 -

Develop a self-sustaining, solar-powered Wireless Sensor Network (WSN) for real-time soil health monitoring and crop yield optimization.

Description:

Large-scale farms require continuous monitoring of soil metrics, but running cables or constantly replacing batteries in remote sensor nodes generates agricultural e-waste and increases maintenance. Develop an ultra-low-power embedded system that harvests solar energy to power soil sensors, transmitting data bursts over long distances to a central farm gateway. This creates a zero-maintenance, real-time tracking grid for smart farming.

The solution could focus on one or more of the following:

- **Energy Harvesting & Deep Sleep:** Implement a power management circuit using mini solar panels and lithium-ion batteries, programming the microcontroller to enter deep sleep between readings to conserve power.
- **Sensor Integration:** Interface capacitive soil moisture sensors, temperature, and humidity modules to accurately log the micro-climate of the soil.
- **Long-Range Communication:** Utilize LoRa (Long Range) or Zigbee transceiver modules to establish a mesh or star network capable of transmitting data packets across large farm areas without Wi-Fi.
- **Localized Data Dashboard:** Build a local gateway (e.g., using a Raspberry Pi or ESP32) that receives the network data and displays it on an offline, user-friendly LCD or a local web server for the farmer.

PROBLEM STATEMENT 2 -

Design a decentralized smart irrigation controller that interfaces with off-grid solar micro-grids to prevent inverter overload and optimize water usage.

Description:

Rural farms utilizing off-grid solar panels frequently experience power trips when high-wattage water pumps are activated during low sunlight or peak load times. Design a smart, edge-computing control unit that monitors real-time power generation and schedules irrigation dynamically based on available solar capacity, battery health, and localized soil moisture levels.

The solution could focus on one or more of the following:

- **Active Power Monitoring:** Use voltage and current sensor modules (like INA219) to continuously measure the power being generated by the farm's solar array and the power consumed by the grid.
- **Dynamic Load Balancing:** Develop an algorithm that triggers heavy DC water pumps via multi-channel relays only when excess solar power is available or when soil moisture drops to a critical emergency threshold.
- **Fault Detection & Safety Override:** Implement hardware-level safety mechanisms that instantly cut power to the pumps if a short circuit, voltage drop, or dry-run condition is detected.
- **Wireless Actuation & Alerts:** Provide a Bluetooth (BLE) or Wi-Fi interface allowing farmers to manually override the system and receive low-power alerts on their smartphones.

PROBLEM STATEMENT 3 -

Create an autonomous UAV (drone) RF-payload system that acts as a "Data Mule" to retrieve environmental data from isolated, low-power farm sensors.

Description:

In vast agricultural zones lacking Wi-Fi or cellular networks, standard IoT gateways fail to collect distributed sensor data. Build a specialized, lightweight hardware payload for a drone. As the drone flies its automated route over the field, the payload securely harvests data packets from isolated ground nodes via short-range RF, logging them onboard to be synchronized later at the base station.

The solution could focus on one or more of the following:

- **Drone Payload Integration:** Design a compact, battery-powered circuit using a microcontroller and an SD card module that is light enough to be mounted on a standard commercial or DIY drone.
- **RF Data Harvesting:** Use transceiver modules (like nRF24L01 or LoRa) on both the ground sensors and the drone to initiate rapid, handshake-based data transfer when the drone flies within range.
- **Ground Node Synchronization:** Program the ground sensors to buffer their daily readings locally and stay in low-power mode until they receive a specific "wake-up" ping from the approaching drone.
- **Base Station Data Sync:** Create a simple interface where the drone payload automatically dumps its collected CSV data to a master computer once it returns to the charging pad.

PROBLEM STATEMENT 4 -

Develop an edge-AI (TinyML) driven bio-mimetic climate control system for greenhouses to minimize grid-power consumption.

Description:

Greenhouses consume massive amounts of grid power by running continuous HVAC systems instead of utilizing natural environmental shifts. Develop a closed-loop hardware controller that uses a lightweight machine learning model deployed directly on a microcontroller. The system should predict micro-climate changes and actuate low-energy mechanisms before engaging high-power cooling or heating.

The solution could focus on one or more of the following:

- **Edge AI Prediction:** Train a TinyML model (using platforms like Edge Impulse) on a microcontroller (e.g., Arduino Nano 33 BLE) to recognize patterns in temperature, humidity, and CO2 data to predict upcoming heat spikes.
- **Low-Power Actuation:** Use servo motors to automatically open physical roof vents or trigger low-power water misting relays to cool the greenhouse naturally based on the AI's predictions.
- **Heavy-Load Fallback:** Ensure the system acts as a tiered controller, only activating heavy grid-powered exhaust fans when natural, low-power actuation fails to regulate the temperature.
- **MQTT Telemetry & Dashboard:** Transmit the environmental logs and actuation history over an MQTT broker to a visual dashboard, allowing operators to see exactly how much grid energy the AI has saved.

PROBLEM STATEMENT 5 -

Design a low-power, wearable bio-sensor to monitor physiological stress markers and pesticide neurotoxicity exposure in agricultural workers.

Description:

Prolonged exposure to certain agricultural chemicals (like organophosphate pesticides) is scientifically linked to neurological distress, severe depression, and increased risk of self-harm. Furthermore, extreme physical exhaustion often precedes mental health crises. Design a rugged, solar-rechargeable wearable device tailored for farmers that continuously monitors physiological stress and environmental toxicity, providing immediate haptic feedback and remote alerts when dangerous thresholds are crossed.

The solution could focus on one or more of the following:

- **Toxicity Detection:** Integrate a miniaturized VOC (Volatile Organic Compound) or gas sensor module to detect high concentrations of harmful pesticide fumes in the farmer's immediate breathing zone.
- **Stress & Fatigue Monitoring:** Use Galvanic Skin Response (GSR) and heart-rate sensors to track abnormal spikes in physiological stress or severe sleep deprivation over several days.
- **Local Mesh Alerts:** When the wearable detects sustained toxic exposure or critical stress patterns, it transmits an encrypted alert via Zigbee or BLE to a home receiver, notifying family members to step in.

Problem Statement 6 -

AI-Based Groundwater Monitoring and Smart Water Distribution System for Rural India

Background:

Groundwater serves as the primary source for drinking and irrigation across rural India, but it faces severe depletion due to over-extraction and erratic monsoons. National initiatives like the Jal Jeevan Mission and Atal Bhujal Yojana mandate community-led water security, yet local bodies still rely on sporadic, manual well inspections. This data void leads to unannounced borewell failures and highly inequitable water distribution across rural hamlets. Consequently, there is an urgent administrative requirement for an affordable, localized IoT and analytical system to enable real-time water auditing and data-driven equitable supply management.

Description:

The fundamental challenge in rural water resource management is the critical absence of continuous spatio-temporal data regarding water table dynamics and qualitative degradation. Because aquifer behavior varies significantly based on local hydrogeological frameworks, soil strata, and localized extraction vectors, static empirical rules fail to forecast depletion or contamination events.

This infrastructure gap manifests operationally as sudden pump failures during peak demand cycles, unmonitored exposure to chemical contaminants, and asymmetrical water allocation among shared rural households. Implementing a localized edge-computing sensor node paired with deterministic predictive models allows for continuous asset health tracking, early anomaly generation, and structural decision support, empowering Gram Panchayats with actionable insights for equitable resource distribution.

Expected Solution:

The solution could focus on one or more of the following:

- A sensor node built using ESP32/Arduino integrated with a waterproof ultrasonic level sensor (e.g., JSN-SR04T) and a basic water quality sensor (TDS or pH), mounted at a well, borewell, or overhead tank.
- Data transmission over WiFi/GSM to a laptop or Raspberry Pi for processing, with local SD card logging as a fallback for areas with intermittent connectivity.
- Simple trend analysis (moving average or linear regression using Python/Pandas) on historical or simulated level data to detect a declining pattern and estimate the time remaining before a critical low level is reached.
- Threshold-based alert logic that triggers a local buzzer/LED and a notification on a console or webpage when water level or quality crosses a defined safe limit.

- A basic dashboard showing current readings from one or more monitored points, along with a simple rule-based recommendation (e.g., reduce allocation to a zone if its source level falls below a set threshold) to support fair distribution.

Problem Statement 7:

LastMile Alert – Resilient Offline Emergency Warning Dissemination for Rural Communities

Background:

When severe floods, cyclones, or heat waves hit rural communities, early warnings exist but they stop right where the internet ends.

During a major disaster, the tech infrastructure we take for granted completely collapses. Cellular towers lose power, mobile networks get choked, and the internet goes down. This creates a dangerous silence. The very people who need warnings the most - local volunteers, village leaders, and families living in high-risk zones are left totally cut off. They don't have access to high-end smartphones or stable data plans even on a good day.

Standard warning apps are completely useless the moment the network drops. We are facing a critical bottleneck: life-saving evacuation and safety data cannot cross the "last mile" into a disconnected village. The challenge isn't just about predicting the weather; it is about building a system rugged enough to deliver that data through the dark when every traditional communication line fails.

Description:

The core of this challenge is a structural flaw: our current warning systems rely entirely on an "always-on" internet connection. When extreme weather strikes, networks get heavily congested or shut down completely, cutting off access to evacuation routes, flood levels, and shelter locations.

To solve this, we need a resilient, software-driven communication system that refuses to rely on the internet alone. The system should act as an intelligent routing engine, automatically jumping to whatever communication channel is still standing (whether that is cellular data, basic SMS text, a local offline Wi-Fi network, or peer-to-peer device sharing). It must compress complex government alert data into ultra-lightweight payloads that can travel across weak networks. Furthermore, because it is designed for rural communities, the interface must handle multiple regional languages, remain readable for users with low digital literacy, and consume very little phone battery. By leveraging local offline caching and device-to-device sharing, this solution will ensure that a village is never blinded at the moment it matters most.

Expected Solution

The solution could focus on one or more of the following:

- **Intelligent Routing Engine:** Develop a smart network state machine that automatically detects a phone's connection status. The system must seamlessly transition between available channels, prioritizing the fastest option but automatically falling back down the line: Internet - Local Offline Wi-Fi LAN - SMS / P2P Simulation without crashing or throwing error screens.
- Offline Propagation & Store-and-Forward Queues: Ensure alerts can travel even during a total cellular blackout. Implement a local synchronization mechanism (such as using a local Wi-Fi hotspot router or a peer-to-peer framework wrapper) that lets one device push its cached alert data to a nearby device that hasn't synced yet.
- **Secure Local Storage Layer:** Build a rugged offline database (using IndexedDB, SQLite, AsyncStorage, or equivalent) on the device. It must store active alerts, evacuation maps, shelter locations, and emergency guidelines locally, indexing them by hazard type and severity color codes (IMD's Green/Yellow/Orange/Red) so users can instantly access data without a cell signal.
- **Dual-Interface Ecosystem:** Design two distinct, clean user flows:
- **The Authority Portal:** A control dashboard where a local official or Panchayat leader can view incoming alert feeds, see which local devices have synced, or manually trigger a highly localized alert for an unlisted emergency.
- **The Citizen Interface:** A minimal, high-contrast app optimized for low-end Android phones (Android 8+ / 2GB RAM) where villagers can view active warnings, hit an "Acknowledge Alert" button to confirm safety, and queue up an offline "Report Status" update that automatically sends out the moment the phone re-detects a signal.
- **Official Feed Integration:** Write a clean backend pipeline to parse incoming emergency data. Your system should ingest structured alert feeds from standard API formats (such as IMD's GeoJSON weather warnings, CWC flood bulletins, or mock NDMA Common Alerting Protocol datasets) and transform them into a standardized format for the app.
- **Accessible, Multilingual UX:** Create a user experience that prioritizes clarity under panic. The system must convert technical descriptions into simple, 2–3 line plain-language summaries in at least one regional language (e.g., Hindi, Odia, Bengali). Pair these text summaries with highly visual, color-coded icons designed specifically for low-literacy users.

Technical Note for Participants: For the peer-to-peer and local wireless communication requirements, teams are not expected to write low-level radio or hardware mesh code from scratch. You can fully fulfill this criteria using high-level software simulations such as establishing an offline local Wi-Fi router network, using local WebSocket connections, utilizing near-field device APIs (like Google Nearby), or creating visual data transfers using dynamic, data-dense QR codes scanned from screen to screen.

Problem Statement 8 -

Smart Livestock Health Monitoring Wearable for Early Disease Detection

Background:

In rural India, most smallholder farmers manage livestock through direct observation alone and by the time an animal visibly looks sick, the disease has often already progressed significantly. Illnesses like Foot-and-Mouth Disease (FMD), Lumpy Skin Disease, and Mastitis can devastate a family's livelihood within days if caught late. With veterinary access sparse across rural blocks and national programs like NADCP and NLM pushing for tech-driven animal health solutions, there is a real, urgent gap: farmers need an affordable, always-on tool that can flag trouble before it becomes a crisis.

Description:

The frustrating reality is that most livestock diseases leave detectable physiological traces - a rising body temperature, irregular movement, slower rumination well before any visible symptom appears. But farmers check on animals once or twice a day at best, and a trained vet might be 20–30 km away. By the time something looks wrong, the window for easy intervention has often already closed, and infection may have spread through the herd. A simple wearable embedded with bio-sensors, a microcontroller, and a wireless alert module can change this entirely - continuously tracking vitals, flagging anomalies on-device, and pushing an SMS or app alert to the farmer and the nearest para-vet while treatment is still straightforward.

Expected Solution:

Teams must design and prototype a livestock wearable health monitoring device that attaches non-invasively to an animal (cattle/goat) ear-tag, neck collar, or leg band and performs the following:

- Measure at least two vital parameters continuously: body temperature (IR or contact thermistor) and physical activity/posture (accelerometer such as MPU-6050)
- Use a microcontroller (ESP32 or Arduino Nano) to process sensor data locally
- Define threshold values (e.g., temperature > 39.5°C = fever) and trigger a visual/audio alert (LED/buzzer) on breach
- Transmit alert data wirelessly (Wi-Fi, Bluetooth, or GSM SIM800L) to a mobile app or simple web dashboard
- Demonstrate live with real-time sensor readings
- Add a heart rate or rumination sensor as a third vital parameter
- Implement a rule-based or lightweight ML anomaly model running on-device or edge server
- Design a low-power mode with 48–72 hour battery life for field use

- Send SMS alerts to the farmer's phone via GSM , no internet required
- Submit a short technical report covering sensor calibration, threshold rationale, and deployment notes.

Problem Statement 9 :

IoT-Based Smart Beehive Monitoring System for Early Detection of Colony Stress and Disease.

Background:

Apiculture forms an important component of India's agricultural economy, supporting crop pollination alongside rural livelihoods through honey and allied produce. Programmes such as the National Beekeeping & Honey Mission (NBHM) promote the adoption of scientific, technology-enabled apiculture practices among small and marginal beekeepers. Ground-level users are typically individual beekeepers or cooperatives managing multiple hive boxes across farmland and orchard sites, with limited time available for regular inspection. Colony health is conventionally assessed through direct physical inspection, a process that is invasive, time-consuming, and disruptive to normal hive activity. As such inspections can only be conducted periodically, early indicators of colony stress or disease frequently go unnoticed until the colony's condition has deteriorated significantly.

Description:

Early-stage indicators of colony stress or disease, including Varroa mite infestation, foulbrood, Nosema, queenlessness, or impending swarming, rarely present as a single identifiable symptom, instead surfacing as correlated deviations in brood-nest temperature, humidity, acoustic signature, and hive weight. As no single parameter is conclusive in isolation, and periodic manual inspection cannot capture such gradual shifts, these indicators are frequently missed until the colony's condition has visibly deteriorated, resulting in population decline, reduced honey yield, or complete collapse, with consequent loss of pollination coverage for surrounding farmland. A non-invasive embedded system that continuously monitors these parameters and flags deviation from a healthy baseline can enable timely, targeted intervention without disrupting the colony.

Expected Solution:

The solution could focus on one or more of the following:

- A hive-mounted sensor node built around a low-power microcontroller (e.g., ESP32) interfacing:
- Temperature & humidity sensor (DHT22/SHT31) positioned near the brood chamber
- Load cell + HX711 amplifier module under the hive stand for continuous weight tracking
- A sound sensor / electret microphone module to capture the colony's acoustic signature

- Periodic data logging (every 10–15 minutes) with wireless transmission (WiFi/Bluetooth) to a cloud service or local gateway
- A simple web/mobile dashboard displaying live readings and historical trends per hive
- Rule-based anomaly detection e.g., flag if temperature deviates from the 34–35°C brood range, weight drops beyond a set threshold within a short window, or sound amplitude deviates from the baseline
- Real-time alerting via SMS, app notification, or an on-site buzzer/LED when thresholds are breached
- A weatherproof, bee-safe enclosure suitable for outdoor apiary deployment

OPTIONAL Add-on Features:

- Solar + battery power management for fully autonomous, off-grid operation
- On-device (TinyML) or cloud-based acoustic classification to distinguish normal foraging hum from queenlessness, swarming, or distress signals, instead of simple amplitude thresholds
- Multi-hive network using LoRa/LoRaWAN with a central gateway for apiaries with several colonies
- Correlating weight/temperature trends with external weather data (via API) to help separate disease-driven decline from natural nectar dearth
- A composite "colony health score" combining all sensor streams into one beekeeper-friendly indicator
- Optional low-cost camera module at the hive entrance for bee traffic/activity counting as a supplementary stress indicator

Problem Statement 10 -

Vibration-Based Predictive Maintenance System for Farm Equipment.

Background:

Farm equipment such as tractors, pump sets, threshers and tillers is widely used across Indian agriculture, with a significant share operated through Custom Hiring Centres (CHCs) under the Sub-Mission on Agricultural Mechanization (SMAM). Sudden equipment failure during critical agricultural operations leads to operational delays and direct financial loss to farmers and CHC operators. Maintenance practices currently followed are largely reactive or based on fixed servicing intervals, rather than the actual condition of the machine. Early indicators of mechanical wear, such as bearing degradation or misalignment, typically go undetected due to the absence of any condition-monitoring mechanism at the field level. A need therefore exists for an affordable, field-deployable solution capable of providing early warning of impending equipment failure.

Description:

Mechanical faults in rotating machinery generally develop gradually, with early-stage changes in vibration and acoustic signatures appearing well before the failure becomes audible or visible to the operator. Reliable detection of such faults is difficult because vibration characteristics vary across machine types, operating loads and usage conditions, making a single fixed threshold ineffective for identifying abnormal behaviour. In the absence of a monitoring mechanism, faults are typically identified only after the equipment has already failed, resulting in loss of operational days during critical periods such as sowing, irrigation and harvesting, along with higher expenditure on emergency repair compared to planned maintenance. This problem can be addressed through a low-cost, sensor-based monitoring solution that applies machine learning to vibration and acoustic data, establishes the normal operating signature of a given machine, and identifies deviations at an early stage, enabling preventive maintenance instead of reactive repair.

Expected Solution:

The solution could focus on one or more of the following:

- A sensor unit built using ESP32/Arduino integrated with an accelerometer module (MPU6050/ ADXL345), mounted on a motor, pump or fan to record vibration data during operation.
- Transmission of sensor data to a laptop or Raspberry Pi via WiFi, Bluetooth or a serial connection for processing; on-chip inference is not mandatory at this stage.
- Extraction of basic time-domain features (mean, RMS, standard deviation, peak amplitude) from the vibration signal using standard Python libraries such as NumPy and Pandas.
- A binary classification model (Decision Tree, Logistic Regression or SVM) trained on a publicly available bearing-fault dataset such as the CWRU Bearing Dataset, to distinguish between "Normal" and "Fault" states.
- A basic alerting mechanism, such as an LED or buzzer connected to the microcontroller, or a simple notification on a console/application, indicating when a fault is detected.

OPTIONAL Add-on Features:

- Frequency-domain analysis using FFT to improve detection accuracy over time-domain features alone.
- Classification of specific fault types (e.g., bearing wear, imbalance, looseness) instead of binary detection.
- On-device inference using TinyML/Edge Impulse in place of laptop or Raspberry Pi-based processing.
- A basic dashboard or mobile interface displaying equipment health status and alert history.